K.R. MANGALAM UNIVERSITY

Unit–3 Assignment: Sorting Algorithms

SUBMITTED BY- Sree Parvati

ROLL NO - 2501730236

COURSE - Data Structures

PROGRAMME - B. Tech CSE (AI & ML)

SECTION - A

SUBMITTED TO - Mrs. Neetu Chauhan

1.Correctness Check

Before measuring performance, all algorithms were tested on small input.

Algorithm	Output
Insertion Sort	[1,2,5,5,6,9]
Merge Sort	[1,2,5,5,6,9]
Quick Sort	[1,2,5,5,6,9]

All algorithms produced the correct sorted output.

2.Execution Time Results

Size | Type | Insertion(ms) | Merge(ms) | Quick(ms)

1000 | Random | 14.31 | 3.68 | 1.14

1000 | Sorted | 0.08 | 0.96 | 48.44

1000 | Reverse | 31.17 | 1.05 | 27.54

5000 | Random | 482.19 | 7.99 | 4.26

5000 | Sorted | 0.37 | 7.76 | 871.87

5000 | Reverse | 832.97 | 6.06 | 638.59

10000 | Random | 1682.87 | 18.89 | 9.53

10000 | Sorted | 1.11 | 11.27 | 3449.85

10000 | Reverse | 3234.30 | 12.27 | 2498.53

3.Analysis and Comparison

1. <u>Random Input</u>

- Quick Sort performed the fastest in most cases.
- Merge Sort showed stable and consistent performance.
- Insertion Sort was much slower due to its $O(n^2)$ complexity.

This matches theory:

- Quick & Merge → **O(n log n)**
- Insertion → **$O(n^2)$**

2. <u>Sorted Input</u>

- Insertion Sort performed extremely fast (almost O(n)) because the list was already sorted.
- Merge Sort maintained stable performance.
- Quick Sort performed very poorly.

Reason:
Quick Sort used last element as pivot, causing worst-case behavior → $O(n^2)$.

3. <u>Reverse Sorted Input</u>

- Insertion Sort became very slow because maximum shifting was required.
- Merge Sort again remained stable.
- Quick Sort performed poorly, similar to sorted input.

Both Insertion and Quick show $O(n^2)$ behavior in this case.

4.Complexity Discussion

Algorithm	Best Case	Average Case	Worst Case
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$
Merge Sort	O(n log n)	O(n log n)	O(n log n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$

Special Note on Quick Sort

Quick Sort performance depends heavily on pivot selection.

- Using the last element as pivot leads to worst-case **$O(n^2)$** for sorted and reverse-sorted inputs.
- This explains the very high execution times observed in those cases.

5.Stability and Memory

Algorithm	Stability	Memory Usage
Insertion Sort	Stable	In-place
Merge Sort	Stable	Uses extra space
Quick Sort	Not stable	In-place

6.Conclusion

This assignment showed how theoretical time complexity matches practical performance. Merge Sort remained consistent across all inputs, while Quick Sort was fastest for random data but performed poorly in worst-case scenarios. Insertion Sort was efficient only for small or already sorted data. Overall, the results highlight the importance of choosing the right algorithm based on input type.

References

- Lecture notes
- GeeksforGeeks
- Python Documentation